

Арена переговоров. Сопроводительная документация

ЛЦТ-2026, задача №9 "Арена переговоров" (ОЭЗ "Алабуга"), команда "Магнит инноваций".

Репозиторий: <https://github.com/kitkey/innovation-magnet>, ветка `main`. Полная копия в SourceCraft: <https://sourcecraft.dev/lct-hackaton-2026/case-19-negotiation-simulator-team-69>. Развёрнутый прототип: <https://89-169-4-44.sslip.io> (YandexGPT и Yandex SpeechKit). Подробное ТЗ лежит в [docs/TZ.md](#), краткое описание и детали голоса и аватара в [README.md](#).

1. Целевая аудитория, проблема и границы MVP

Кому

Тренажёр рассчитан на молодых специалистов и студентов 17–23 лет на первом месте работы (стажёры, выпускники колледжей и вузов, студенты, которые совмещают учёбу с работой) и на сотрудников проектных команд. Второй пользователь продукта – организация, где они работают. Её администратор готовит сценарии для сотрудников и смотрит их статистику.

Какую проблему решает

Новичок договаривается с руководителем о сроке и объёме задачи, с коллегой о разделении работы, с заказчиком о цене и составе работ. Без навыка переговоров он соглашается на нереальный срок, молчит о важной детали под давлением, берёт чужую задачу или уступает в цене без обоснования. Планового обучения переговорам у этой аудитории обычно нет, разовые тренинги дают теорию без практики и без разбора конкретных реплик. Организация платит за это повторными согласованиями, переделками и потерянными заказчиками.

Арена даёт место, где можно провести такой разговор с LLM-собеседником по своей рабочей ситуации, ошибиться без последствий и получить разбор каждого хода по выбранной методике.

На чём сосредоточились в MVP

В прототипе работают:

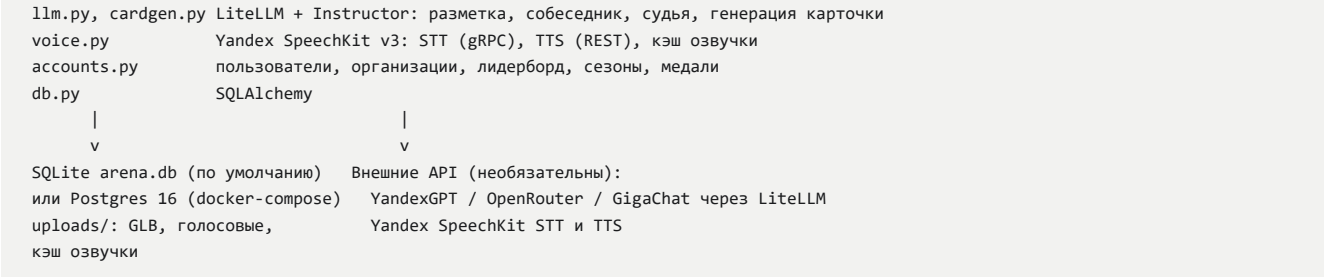
1. Входной тест из восьми ситуаций. Он подбирает стартовую сложность, слабую сторону и первый сценарий.
2. Семь встроенных сценариев и панель, где пользователь или администратор настраивает свой сценарий. Карточку сценария можно собрать LLM по описанию ситуации своими словами.
3. Диалог с собеседником, у которого есть скрытое состояние (доверие, готовность уступить, раздражение). LLM размечает реплику пользователя и формулирует ответ собеседника, а уступку и исход по проверенной разметке рассчитывает детерминированный движок правил.
4. Разбор после диалога от LLM-судьи по осям метода (SPIN, Гарвардский метод, BATNA или свободный режим) с ключевыми моментами и кнопкой "Переиграть отсюда".
5. Голосовые реплики пользователя и 3D-собеседник с озвучкой, липсинком и мимикой по настроению (Yandex SpeechKit и TalkingHead).
6. Маскот-наставник с советами перед диалогом и подсказками по ходу, справочник метода.
7. Аккаунты, организации, кабинет администратора, лидерборд, сезоны и медали.
8. Запасной режим без LLM: разметка, ответы и разбор идут по правилам, внешние платные сервисы не нужны.

В MVP не вошли прогресс пользователя по контрольным точкам, стрики, подписки и оплата, интеграция с HR-системами, оценка связности речи и слов-паразитов. Полный список с пометками "есть в прототипе" или "нет" приведён в разделе 1.3 ТЗ.

2. Архитектура и логика симуляции

Общая схема

```
Браузер: React 19 + Vite, TypeScript
+ экраны: тест, сценарии, настройка, диалог, разбор, история, лидерборд, организация
+ запись голоса: MediaRecorder -> WAV 16 кГц
+ 3D-собеседник: TalkingHead (three.js) + русский липсинк
  |
  | HTTP/JSON: /api/*, статика /uploads/*
  v
Бэкенд: Python, FastAPI, uvicorn (backend/app/main.py)
service.py      ход пользователя по шагам:
    1) разметка хода      -> llm.py или engine/offline.py
    2) страховки разметки -> engine/guards.py
    3) счётчики и исход   -> engine/rules.py (без LLM)
    4) реплика собеседника -> llm.py или шаблон
    5) подсказки маскота  -> engine/hints.py (без LLM)
```



LLM-часть устроена как мультиагентная система: четыре агента с отдельными ролями и отдельной настройкой модели. Генератор собирает карточку сценария из описания, разметчик размечает ход игрока, собеседник отвечает от лица персонажа, судья разбирает сессию. Уступки и исход при этом считают правила `engine/rules.py`, агенты на них напрямую не влияют.

В Docker-сборке фронтенд собирается в статику и отдаётся тем же процессом FastAPI на порту 8000. При локальной разработке фронтенд работает на Vite-сервере (порт 5173) и проксирует `/api` и `/uploads` на бэкэнд.

Где что лежит

Компонент	Код	Что делает
API	backend/app/main.py, accounts.py	REST-эндпоинты сценариев, сессий, голоса, аккаунтов и организаций; отдаёт собранный фронтенд
Сессия и ход	backend/app/service.py	ведёт сессию, пишет ходы в БД, переигровка с выбранного хода
Движок правил	backend/app/engine/rules.py	счётчики собеседника, уступки, условия завершения
Страховки разметки	backend/app/engine/guards.py	сверяют метки LLM, от которых зависит исход, с текстом реплики: число предложения только из слов игрока, вопрос не бывает предложением или согласием, согласие и выход только по явным словам
Режим без LLM	backend/app/engine/offline.py	разметка по словам-маркерам, шаблонные ответы, разбор по правилам
Подсказки маскара	backend/app/engine/hints.py	условия по разметке и счётчикам
LLM	backend/app/llm.py, cardgen.py	классификатор хода, реплика собеседника, судья, генерация карточки
Очки и рейтинг	backend/app/scoring.py	очко сессии, рейтинг, медали
Голос	backend/app/voice.py, backend/app/yandex/	распознавание и озвучка через SpeechKit v3
Данные	backend/app/db.py, schemas.py, seeds.py	таблицы, схемы, встроенные сценарии
Фронтенд	frontend/src/	экраны, avatar/ (TalkingHead и липсинк), voice/ (запись)

Схема БД (таблицы `orgs`, `seasons`, `medals`, `users`, `auth_tokens`, `scenarios`, `sessions`, `turns`) описана в разделе 4.7 ТЗ. Таблицы создаются при старте, миграций нет.

Как проходит один ход

- Пользователь пишет реплику или отправляет голосовое. Голосовое распознаёт SpeechKit, дальше текст идёт как обычная реплика.
- Классификатор хода (один вызов LLM со структурированным ответом через Instructor) размечает реплику метками приёмов: вопрос об интересах, вопросы SPIN, объективный критерий, конкретное предложение, уступка, эмпатия, давление, переход на личности, манипуляция, сравнение с альтернативой, граница, условный обмен, выход к альтернативе и др. Он же возвращает озвученные обязательные детали и значение, которое предложил пользователь. Страховки из `engine/guards.py` снимают метки, которых в тексте реплики нет.
- Движок правил сдвигает счётчики собеседника по таблице эффектов меток. Стиль собеседника умножает сдвиг (у жёсткого доверие растёт вдвое медленнее, у эмоционального раздражение вдвое быстрее).
- Движок решает, уступает ли собеседник. Уступка возможна, когда готовность дошла до порога сложности (лёгкая 3, средняя 4, сложная 5) и доверие не ниже нуля. Одна уступка сдвигает позицию собеседника на четверть пути к его пределу, предел зависит от сложности и лежит в целевой зоне.
- LLM пишет реплику собеседника, зная решение движка, его текущую позицию, стиль и скрытые интересы. Если в ответе модель назвала число лучше разрешённой позиции, бэкэнд заменяет ответ шаблонным.
- Бэкэнд проверяет условия завершения и подсказки маскара и возвращает ход фронтенду вместе со шкалами "Доверие", "Терпение" и "Готовность уступить".

Одинаковая разметка ходов даёт одинаковый исход. Поэтому переигровка с выбранного хода пересчитывает состояние по сохранённой разметке без вызовов LLM.

Как заканчивается сессия и как считается оценка

Переговоры заканчиваются соглашением, рациональным выходом к альтернативе, срывом (раздражение дошло до порога сложности или доверие упало до -6), лимитом ходов или кнопкой "Завершить". Выход к альтернативе засчитывается верным, если позиция собеседника для пользователя хуже, чем худший для него край целевой зоны.

Судья (отдельный вызов LLM) получает расшифровку с разметкой, карточку и скрытые интересы собеседника и ставит оценки 0–100 по четырём осям метода, выбирает 1–3 ключевых момента в формате "цитата → что не так → как сказать лучше". Цитаты, которых нет в репликах пользователя, бэкэнд отбрасывает. Очко сессии равно средней оценке по осям, умноженной на множитель сложности (0,8, 1,0 или 1,25), плюс 10 за соглашение в целевой зоне или верный выход к альтернативе. Рейтинг в лидерборде считается по пяти лучшим сессиям за период.

Встроенные сценарии

Сценарий	Метод	Сложность	Предмет торга	Собеседник
Срок отчёта с руководителем	Гарвардский	средняя	срок, рабочих дней	Сергей Власов
Цена и объём работ с заказчиком	SPIN	средняя	стоимость проекта, тыс. руб.	Артур Галиев
Разделение работы с коллегой	Гарвардский	лёгкая	число задач из 12	Анна Лаптева
Аренда цеха в ОЭЗ "Алабуга"	SPIN	сложная	скидка к ставке аренды, %	Марина Хитрова
Поставщик повышает цену	BATNA	средняя	рост цены поставки, %	Ольга Кравец
Переговоры об оффере	BATNA	сложная	прибавка к окладу, %	Дарья Шувалова
Пункт о допработах в договоре	BATNA	средняя	формулировка пункта (варианты)	Олег Ситников

Что настраивается

Карточка сценария задаётся в панели "Новый сценарий" или "Настроить" (поля перечислены в разделе 4.1 ТЗ). В ней задаются сфера и тема, сложность, тон, лимит ходов (3–30), тренируемый метод, стиль собеседника (жёсткий, сотрудничающий, уклоняющийся, давящий, эмоциональный), роли и цели сторон, скрытые интересы собеседника, альтернативы (BATNA) обеих сторон, целевая зона, первая реплика собеседника, обязательные детали, имя, голос и 3D-персонаж собеседника, наставления и подсказки маскота.

Целевая зона задаётся числом (единица торга, стартовые позиции сторон и края зоны) или вариантами, когда спорят о формулировке условия. Во втором случае в карточке от 2 до 8 формулировок от лучшей для пользователя к худшей, и движок работает с номерами вариантов так же, как с числами.

Кнопка "Сгенерировать карточку" собирает карточку по описанию ситуации своими словами. Бэкэнд проверяет результат, мелкие огрехи исправляет сам, остальные возвращает модели на повторную попытку. Если карточка так и не собралась, форма показывает причину.

Администратор организации может зафиксировать сценарий, и сотрудники проходят его без права правки. Обычные пользователи встроенные сценарии не меняют: их можно скопировать и настроить копию. Владелец `ADMIN_TOKEN` может править и оригиналы.

Запасной режим без LLM

При `OFFLINE_MODE=true` тренажёр целиком работает на правилах из `engine/offline.py`. Если LLM не ответила на отдельном вызове, правила заменяют только этот этап: разметку, реплику или разбор. Метки ставятся по словам-маркерам, собеседник отвечает шаблонами, судья считает оценки по доле подходящих меток по каждой оси, и на экране разбора появляется пометка "Разбор по правилам (LLM недоступна)". Движок уступок и исходов тот же самый, поэтому механика сценариев в этом режиме не меняется. Генерация карточки по описанию без LLM недоступна.

3. Сборка, запуск и демонстрация

Что нужно

- Docker с Docker Compose 2.24 или новее (для варианта А) или Python 3.12+ и Node.js 20.19+ или 22.12+ (для варианта Б). Проверено на чистом клоне с Python 3.12, Node 24 и Docker 29 (Windows 11).
- Браузер Chrome, Firefox, Edge или Safari актуальной версии. Для 3D-режима нужен WebGL, для голосовых реплик микрофон.

Зависимости зафиксированы: `backend/requirements.txt` с точными версиями, `frontend/package-lock.json`.

Настройки

Все настройки читаются из файла `.env` в корне репозитория. Шаблон лежит в `env.example`:

```
cp env.example .env          # Windows PowerShell: Copy-Item env.example .env
```

Для запуска без ключей и платных сервисов достаточно поставить в `.env` строку `OFFLINE_MODE=true`. Для работы с LLM впишите модель и ключ (раздел 5).

Вариант А. Docker Compose

```
docker compose up --build
```

Откройте <http://localhost:8000>. Compose поднимает Postgres 16 и приложение, данные и загрузки хранятся в томах `pgdata` и `uploads` и переживают перезапуск. Если порт 8000 занят, поменяйте левую часть `"8000:8000"` в `docker-compose.yml`.

Вариант Б. Без Docker

Нужны два терминала. Бэкенд по умолчанию хранит данные в SQLite, в файле `backend/arena.db`.

Терминал 1, бэкенд. Windows PowerShell:

```
cd backend
python -m venv ../.venv
../.venv/Scripts/Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

Если PowerShell запрещает запуск `Activate.ps1`, выполните один раз `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` или вызывайте интерпретатор окружения напрямую (`../.venv/Scripts/python.exe -m pip ...`, `../.venv/Scripts/python.exe -m uvicorn ...`).

Linux и macOS:

```
cd backend
python3 -m venv ../.venv
source ../.venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

Терминал 2, фронтенд (одинаково на всех системах):

```
cd frontend
npm ci
npm run dev
```

Откройте <http://localhost:5173>. Vite проксирует запросы `/api` и `/uploads` на порт 8000.

Бэкенд можно запустить и одним процессом вместе с фронтендом, как в Docker. Для этого соберите фронтенд и положите сборку в `backend/static`:

```
cd frontend
npm ci
npm run build
mkdir -p ../backend/static && cp -R dist/. ../backend/static/ # Windows PowerShell: Copy-Item -Recurse -Force dist\* ../backend/static
```

После этого `uvicorn app.main:app --port 8000` из папки `backend` отдаёт приложение на <http://localhost:8000>.

Проверка, что всё поднялось

```
curl http://localhost:8000/api/health
# {"ok":true,"offline_mode":true,"model":"..."}
curl -X POST "http://localhost:8000/api/sessions?scenario_id=deadline-with-manager"
# {"session_id":"...","opening":"Насчёт вашего письма про срок...","max_turns":10}
curl -X POST http://localhost:8000/api/sessions/<session_id>/turn \
-H "Content-Type: application/json" \
-d '{"message":"Что для вас важно в этом отчёте к совещанию?"}'
# {"turn":1,"opponent_message":"...","status":"active","analysis":{"labels":["interest_question"],...},"state":{"..."}}
```

Интерактивная документация API открывается на <http://localhost:8000/docs>.

Тесты бэкенда работают в режиме без LLM на временной SQLite и не трогают `arena.db`:

```
cd backend
python -m pytest
```

Демонстрация за 5 минут

Сценарий для показа – "Срок отчёта с руководителем" (Гарвардский метод, средняя сложность). Всё ниже работает и в режиме `OFFLINE_MODE=true`.

1. При первом открытии появляется входной тест. Нажмите "Пропустить тест" или ответьте на восемь вопросов и посмотрите рекомендованную сложность и сценарий.
2. На странице "Сценарии" найдите группу "Работа в команде". У встроенного сценария кнопка "Сделать копию" открывает панель настройки с заполненной карточкой. Там видны метод, стиль собеседника, целевая зона (12, 8–10, 5 рабочих дней), скрытые интересы, галерея персонажей и кнопки "?" у полей. "Сохранить и начать" сохраняет копию и открывает диалог. Можно вернуться и нажать "Начать" у исходного сценария.
3. В диалоге маскот показывает наставления, последняя кнопка "Начать переговоры". Собеседник открывает разговор первой репликой. Справа бриф с ролью, целью и альтернативой пользователя, слева сцена собеседника и шкалы.
4. Сделайте 3–4 хода и посмотрите на метки приёмов под каждой репликой и сдвиги шкал. В режиме без LLM эти реплики дают такие метки: "Что для вас важно в этом отчёте к совещанию?" (вопрос об интересах, растёт готовность), "Данные от смежного отдела приходят с опозданием, прошлый раз на три дня" (объективный критерий и обязательная деталь, собеседник уступает с 5 до 6 дней), "Предлагаю 9 рабочих дней, а к совещанию сделаю предварительную версию" (конкретное предложение и вторая обязательная деталь), "Вы серьёзно? Сколько можно давить" (давление, доверие падает, раздражение растёт). С LLM разметка точнее и ответы собеседника живые.
5. Переключатель "Текст" / "3D и голос" в шапке включает 3D-собеседника. Без ключа Yandex аватар говорит беззвучно с субтитрами и меткой "Без звука".
6. Нажмите "Завершить". В разборе посмотрите исход, график доверия и готовности уступить, оценки по осям метода, ключевые моменты и балл за сессию.
7. У любого хода нажмите "Переиграть отсюда". Начнётся новая сессия с ходами до выбранного, и можно сыграть этот момент иначе.
8. Вход и организация. По умолчанию всё работает в гостевом режиме, история гостя хранится в браузере. Нажмите "Войти" → вкладка "Регистрация", задайте логин (от 3 символов) и пароль (от 6). В профиле создайте организацию, и вы станете её администратором. В меню появится "Организация" с кодом приглашения, статистикой участников и настройками сезона и медалей. Второй пользователь вступает по коду в своём профиле. Сессии вошедших пользователей попадают в "Лидерборд".

4. Методы, библиотеки и внешние сервисы

Методы

- Переговорные методики SPIN, Гарвардский метод (принципиальные переговоры) и BATNA задают метки приёмов в разметке ходов и оси судьи. Для каждой методики в диалоге есть справочник (`frontend/src/methods.tsx`).
- Детерминированная машина состояний собеседника. Счётчики доверия, готовности и раздражения, пороги по сложности и множители по стилю заданы таблицами в `engine/rules.py`, LLM в решении об уступке не участвует.
- LLM как классификатор и как судья со структурированным выводом. Ответ модели валидируется Pydantic-схемой через Instructor, при ответе не по схеме делается до двух повторов.
- Проверки поверх LLM. Числа в ответе собеседника сверяются с его разрешённой позицией, цитаты судьи с репликами пользователя, сгенерированная карточка с правилами целевой зоны. Реплика пользователя передаётся модели в тегах как данные, попытка манипулировать собеседником размечается отдельной меткой и снижает доверие.
- Правила без LLM для запасного режима: словари маркеров с учётом отрицаний и риторических вопросов, распознавание варианта по словам формулировок.

Библиотеки

Библиотека	Версия	Зачем
FastAPI, uvicorn	0.141.1, 0.53.0	API и веб-сервер
SQLAlchemy	2.1.0	работа с SQLite и Postgres
psycopg	3.3.6	драйвер Postgres
Pydantic, pydantic-settings	2.13.5, 2.15.0	схемы и настройки из <code>.env</code>
LiteLLM	1.102.1	единый вызов любого провайдера LLM, запасные модели
Instructor	1.17.0	структурированный ответ LLM по Pydantic-схеме
grpcio, protobuf	1.84.0, 7.36.2	клиент Yandex SpeechKit STT v3 (код из <code>stt.proto</code> лежит в <code>backend/app/yandex/</code>)
httpx	0.28.1	запросы к SpeechKit TTS
python-multipart	0.0.32	загрузка голосовых и GLB
pytest	9.1.1	тесты
React, React DOM	19	интерфейс
Vite, TypeScript	7.3, 5.9	сборка фронтенда
three.js	0.180	3D-сцена
@met4citizen/talkinghead	1.7	3D-аватар с липсинком (MIT)

Точные версии фронтенда зафиксированы в `frontend/package-lock.json`. Русский модуль липсинка для TalkingHead написан командой (`frontend/src/avatar/lipsync-ru.ts`).

Внешние сервисы

Сервис	Для чего	Обязателен
YandexGPT (Yandex Cloud AI Studio)	реплики собеседника, разметка ходов, судья и генерация карточки (YandexGPT Pro; если в <code>LLM_MODEL</code> указан YandexGPT Lite, эти роли сами переходят на Pro)	нет, без LLM работает режим на правилах
OpenRouter или другой провайдер через LiteLLM	то же, альтернатива YandexGPT	нет
Yandex SpeechKit v3	распознавание голосовых (STT) и озвучка собеседника (TTS)	нет, без ключа ответы текстом, аватар говорит беззвучно

Активы в репозитории: четыре чибби-модели собеседников сделаны в Hunyuan 3D, запасная модель `mpfb.glb` взята из примеров TalkingHead (лицензия CC0), портреты и иллюстрации лежат в `frontend/public`. Во время работы они внешних вызовов не требуют.

5. Ссылки и условия доступа

Ключей в репозитории нет. Все ключи и настройки задаются в `.env` в корне репозитория (шаблон `env.example`) или переменными окружения. `docker compose` читает тот же `.env`.

Переменные окружения

Список соответствует `backend/app/config.py`.

Переменная	По умолчанию	Зачем
OFFLINE_MODE	false	true включает режим на правилах без LLM
LLM_MODEL	openrouter/qwen/qwen3.8-27b:free	основная модель в формате LiteLLM
LLM_API_KEY	пусто	ключ провайдера LLM. Если пусто и LLM_API_BASE указывает на Yandex Cloud, берётся YANDEX_API_KEY
LLM_API_BASE	пусто	адрес API провайдера, для YandexGPT https://llm.api.cloud.yandex.net/v1
LLM_FALLBACKS	две бесплатные модели OpenRouter	запасные модели через запятую, пусто отключает
LLM_LABEL_MODEL	пусто	модель разметки ходов (от неё зависит исход). Если пусто, берётся LLM_MODEL, а YandexGPT Lite заменяется на YandexGPT Pro
LLM_JUDGE_MODEL	пусто	модель судьи, по тому же правилу
LLM_OPPONENT_MODEL	пусто	модель собеседника, по тому же правилу
LLM_CARDGEN_MODEL	пусто	модель генерации карточки, по тому же правилу
LLM_TIMEOUT	60	таймаут вызова LLM, секунд
DATABASE_URL	sqlite:///./arena.db	строка подключения SQLAlchemy. В compose задана на Postgres
ADMIN_TOKEN	пусто	запасной ключ владельца площадки, с ним меняется любой сценарий
INCOMPLETE_SESSION_MIN_TURNS	3	сессия с меньшим числом реплик пользователя остаётся без разбора судьи
YANDEX_API_KEY	пусто	API-ключ сервисного аккаунта Yandex Cloud для SpeechKit (и YandexGPT)
YANDEX_FOLDER_ID	пусто	каталог Yandex Cloud; для API-ключа сервисного аккаунта не нужен
VOICE_TIMEOUT	20	таймаут запросов к SpeechKit, секунд
VOICE_MAX_SECONDS	30	максимальная длина голосового
VOICE_TEXT_NORMALIZATION	false	true включает нормализацию распознанного текста (числа цифрами, пунктуация)
UPLOADS_DIR	backend/uploads	папка для GLB-аватаров, голосовых и кэша озвучки
AVATAR_MAX_MB	30	лимит размера своего GLB
UPLOADS_MAX_MB	1024	общий лимит загрузок

LiteLLM сам читает ключи провайдеров из переменных окружения: OPENROUTER_API_KEY для OpenRouter (compose передаёт её в контейнер), GIGACHAT_CREDENTIALS для GigaChat.

Рекомендуемая конфигурация: YandexGPT и SpeechKit

На этой конфигурации работает демо-сервер команды. В .env указана YandexGPT Lite, а собеседника, разметку ходов, судью и генерацию карточек бэкенд сам переводит на YandexGPT Pro, если LLM_OPPONENT_MODEL, LLM_LABEL_MODEL, LLM_JUDGE_MODEL и LLM_CARDGEN_MODEL пустые. Модели и голос работают на одном API-ключе сервисного аккаунта.

- 1. Создайте сервисный аккаунт в Yandex Cloud: https://yandex.cloud/ru/docs/iam/operations/sa/create
- 2. Назначьте ему на каталог роли ai.languageModels.user (YandexGPT), ai.speechkit-stt.user и ai.speechkit-tts.user (SpeechKit): https://yandex.cloud/ru/docs/iam/operations/sa/assign-role-for-sa
- 3. Создайте API-ключ сервисного аккаунта: https://yandex.cloud/ru/docs/iam/operations/authentication/manage-api-keys
- 4. Узнайте идентификатор каталога (folder_id) в консоли Yandex Cloud и заполните .env :

```
LLM_MODEL=openai/gpt://<folder_id>/yandexgpt-lite/latest
LLM_API_BASE=https://llm.api.cloud.yandex.net/v1
LLM_FALLBACKS=
YANDEX_API_KEY=<API-ключ сервисного аккаунта>
OFFLINE_MODE=false
```

LLM_API_KEY можно оставить пустым, тогда для YandexGPT берётся YANDEX_API_KEY. После перезапуска GET /api/voice/status должен вернуть "stt": true, "tts": true: это значит, что ключ задан. Доступ к SpeechKit проверяется голосовой репликой в режиме "3D и голос". Если ключ не принят, ошибка в чате называет роль, которую нужно проверить. Сервисы Yandex Cloud платные, тарифы: https://yandex.cloud/ru/docs/ai-studio/pricing и https://yandex.cloud/ru/docs/speechkit/pricing

Другие провайдеры

- OpenRouter: ключ на <https://openrouter.ai/keys>, в `.env` `LLM_API_KEY=<ключ>` или переменная окружения `OPENROUTER_API_KEY`. Модель по умолчанию и запасные в `env.example` бесплатные, у бесплатных моделей OpenRouter есть лимиты запросов.
- GigaChat: `LLM_MODEL=gigachat/GigaChat-2-Max` и `GIGACHAT_CREDENTIALS=<ключ авторизации>` (ключ в личном кабинете <https://developers.sber.ru/studio>). В Docker переменную достаточно положить в `.env`, без Docker её нужно экспортировать в окружение перед запуском бэкенда. Конфигурация поддерживается через LiteLLM, с живым ключом команда её не проверяла.
- Локальная модель: любой сервер, который поддерживает LiteLLM (например, Ollama или OpenAI-совместимый сервер), подключается через `LLM_MODEL` и `LLM_API_BASE`.

6. Известные ограничения

- В режиме без LLM разметка идёт по словам-маркерам. Перефразированный приём без маркера может остаться без метки. Ответы собеседника в этом режиме шаблонные, генерация карточки по описанию недоступна.
- Сверка оценок судьи с ручными оценками команды в прототипе не проведена.
- Связность речи, слова-паразиты и длину реплик судья не оценивает, только оси метода.
- Pixar-версии четырёх персонажей остаются в плане развития, в галерее они отмечены "скоро" и не выбираются.
- Голосовые реплики и озвучка работают только с ключом Yandex SpeechKit. WebM и MP3 без ffmpeg на сервере не принимаются, браузерная запись приходит в WAV и ffmpeg не требует.
- Входной тест состоит из восьми вопросов с выбором ответа, формат диалога с LLM остаётся планом. Результат теста хранится только в браузере.
- Прогресса по контрольным точкам, стриков, подписок и оплаты в прототипе нет.
- Конфигурация с GigaChat не проверена с живым ключом.
- Токен входа не имеет срока жизни, миграций БД нет (недостающие колонки добавляются при старте).

План развития

- Проверить работу на GigaChat и добавить анонимизацию: убирать личные данные из реплик перед отправкой во внешнюю модель.
- Адаптация к уровню игрока: по прошлым тренировкам собеседник подбирает тактику и сложность под слабые места игрока.
- Курсы под цели обучения: цель задаёт организация или сам пользователь, прогресс отслеживается по заданиям, тестам и серии дней тренировок.
- Персонажи в стиле Pixar.
- Подписки для частного пользователя и организации со схемой оплаты.